

Курс «Алгоритмы и структуры данных».

2025–2026 учебный год.

Домашнее задание А1.

Анализ строковых сортировок

БПИ246

Прахов Данил

24 мая 2026 г.

Постановка задачи

В работе исследуются стандартные и специализированные алгоритмы сортировки строк. Основная цель — экспериментально изучить влияние общих префиксов строк на производительность различных алгоритмов сортировки.

Исследуемые алгоритмы:

- стандартная сортировка merge sort;
- стандартная сортировка quick sort;
- STRING MERGESORT;
- STRING QUICK SORT;
- MSD RADIX SORT;
- MSD RADIX + STRING QUICK SORT.

1. Генерация тестовых данных

Используемый алфавит:

$A..Z, a..z, 0..9, !@ \% ; \& * () - .$

Параметры генерации:

- длина строк: от 10 до 200 символов;
- размер массива: от 100 до 3000;
- шаг изменения размера: 100.

Типы тестовых наборов:

1. случайные массивы;
2. обратно отсортированные массивы;
3. почти отсортированные массивы;
4. массивы с длинными общими префиксами.

2. STRING MERGESORT

Алгоритм использует длину наибольшего общего префикса (LCP).

Повторное сравнение одинаковых частей строк не выполняется, что уменьшает число по-символьных сравнений.

Теоретическая сложность:

$$O(D + n \log n),$$

где D — суммарное число реально просмотренных символов.

Листинг кода: Полная реализация алгоритма `STRING MERGE SORT` представлена на следующей странице.

```

1 // -----
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 int lcp_compare(const std::string& first,
7               const std::string& second,
8               int start_index) {
9     int first_size = static_cast<int>(first.size());
10    int second_size = static_cast<int>(second.size());
11    int index = start_index;
12
13    while (index < first_size &&
14           index < second_size &&
15           first[index] == second[index]) {
16        ++index;
17    }
18
19    if (index == first_size && index == second_size) {
20        return 0;
21    }
22
23    if (index == first_size) {
24        return -1;
25    }
26
27    if (index == second_size) {
28        return 1;
29    }
30
31    return first[index] < second[index] ? -1 : 1;
32 }
33
34 void merge_sort(std::vector<std::string>& values,
35               int left,
36               int right) {
37     if (right - left <= 1) {
38         return;
39     }
40
41     int middle = (left + right) / 2;
42
43     merge_sort(values, left, middle);
44     merge_sort(values, middle, right);
45
46     std::vector<std::string> merged;
47
48     int first_index = left;
49     int second_index = middle;
50
51     while (first_index < middle &&
52           second_index < right) {
53         if (lcp_compare(values[first_index],
54                       values[second_index],
55                       0) <= 0) {
56             merged.push_back(values[first_index++]);
57         } else {
58             merged.push_back(values[second_index++]);
59         }
60     }
61
62     while (first_index < middle) {
63         merged.push_back(values[first_index++]);
64     }
65
66     while (second_index < right) {
67         merged.push_back(values[second_index++]);
68     }
69
70     for (int index = 0;
71          index < static_cast<int>(merged.size());
72          ++index) {
73         values[left + index] = merged[index];
74     }
75 }
76
77 int main() {
78     std::ios::sync_with_stdio(false);
79     std::cin.tie(nullptr);
80
81     int size;
82     std::cin >> size;
83
84     std::vector<std::string> values(size);
85
86     for (int index = 0; index < size; ++index) {
87         std::cin >> values[index];
88     }
89
90     merge_sort(values, 0, size);
91
92     for (const std::string& value : values) {
93         std::cout << value << '\n';
94     }
95 }

```

3. STRING QUICK SORT

Тернарная быстрая сортировка разбивает массив на три части:

$$< pivot, = pivot, > pivot.$$

Алгоритм эффективно работает при наличии длинных совпадающих префиксов.

Важно: Согласно условиям тестирования, выбор опорного элемента (`pivot`) реализован одинаково как для стандартного `Quick Sort`, так и для строкового `String Quick Sort`. В обоих случаях опорным выбирается первый элемент рассматриваемого подмассива (или центральный, в зависимости от реализации), что обеспечивает объективность замеров.

Листинг кода: Полная реализация алгоритма `STRING QUICK SORT` представлена на следующей странице.

```

1 // -----
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 int char_at(const std::string& value, int depth) {
7     if (depth >= static_cast<int>(value.size())) {
8         return -1;
9     }
10
11     return static_cast<unsigned char>(value[depth]);
12 }
13
14 void string_quick_sort(std::vector<std::string>& values,
15                       int left,
16                       int right,
17                       int depth) {
18     if (left >= right) {
19         return;
20     }
21
22     int lower = left;
23     int greater = right;
24     int pivot = char_at(values[left], depth);
25     int index = left + 1;
26
27     while (index <= greater) {
28         int current = char_at(values[index], depth);
29
30         if (current < pivot) {
31             std::swap(values[lower++], values[index++]);
32         } else if (current > pivot) {
33             std::swap(values[index], values[greater--]);
34         } else {
35             ++index;
36         }
37     }
38
39     string_quick_sort(values, left, lower - 1, depth);
40
41     if (pivot >= 0) {
42         string_quick_sort(values, lower, greater, depth + 1);
43     }
44
45     string_quick_sort(values, greater + 1, right, depth);
46 }
47
48 int main() {
49     std::ios::sync_with_stdio(false);
50     std::cin.tie(nullptr);
51
52     int size;
53     std::cin >> size;
54
55     std::vector<std::string> values(size);
56
57     for (int index = 0; index < size; ++index) {
58         std::cin >> values[index];
59     }
60
61     if (size > 0) {
62         string_quick_sort(values, 0, size - 1, 0);
63     }
64
65     for (const std::string& value : values) {
66         std::cout << value << '\n';
67     }
68 }

```

4. MSD RADIX SORT

MSD RADIX SORT выполняет рекурсивную сортировку, начиная со старшего символа строки.

Теоретическая сложность:

$$O(W \cdot n),$$

где W — средняя длина строки.

Листинг кода: Полная реализация алгоритма MSD RADIX SORT представлена на следующей странице.

```

1 // -----
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 const int ALPHABET_SIZE = 256;
7
8 int char_at(const std::string& value, int depth) {
9     if (depth >= static_cast<int>(value.size())) {
10         return -1;
11     }
12
13     return static_cast<unsigned char>(value[depth]);
14 }
15
16 void msd_sort(std::vector<std::string>& values,
17             std::vector<std::string>& buffer,
18             int left,
19             int right,
20             int depth) {
21     if (right - left <= 1) {
22         return;
23     }
24
25     std::vector<int> count(ALPHABET_SIZE + 2, 0);
26
27     for (int index = left; index < right; ++index) {
28         ++count[char_at(values[index], depth) + 2];
29     }
30
31     for (int index = 1;
32         index < static_cast<int>(count.size());
33         ++index) {
34         count[index] += count[index - 1];
35     }
36
37     std::vector<int> start_positions = count;
38
39     for (int index = left; index < right; ++index) {
40         int bucket = char_at(values[index], depth) + 1;
41         buffer[left + count[bucket]++] = values[index];
42     }
43
44     for (int index = left; index < right; ++index) {
45         values[index] = buffer[index];
46     }
47
48     for (int bucket = 0; bucket < ALPHABET_SIZE; ++bucket) {
49         int begin = left + start_positions[bucket];
50         int end = left + start_positions[bucket + 1];
51
52         if (end - begin > 1) {
53             msd_sort(values, buffer, begin, end, depth + 1);
54         }
55     }
56 }
57
58 int main() {
59     std::ios::sync_with_stdio(false);
60     std::cin.tie(nullptr);
61
62     int size;
63     std::cin >> size;
64
65     std::vector<std::string> values(size);
66
67     for (int index = 0; index < size; ++index) {
68         std::cin >> values[index];
69     }
70
71     std::vector<std::string> buffer(size);
72
73     msd_sort(values, buffer, 0, size, 0);
74
75     for (const std::string& value : values) {
76         std::cout << value << '\n';
77     }
78 }

```

5. MSD RADIX + STRING QUICK SORT

Гибридный алгоритм переключается на STRING QUICK SORT, если размер текущего подмассива становится меньше мощности алфавита.

Гибридный подход уменьшает накладные расходы рекурсии и обеспечивает наиболее стабильную производительность.

Листинг кода: Исходный код гибридного алгоритма MSD RADIX + STRING QUICK SORT представлен на следующей странице.

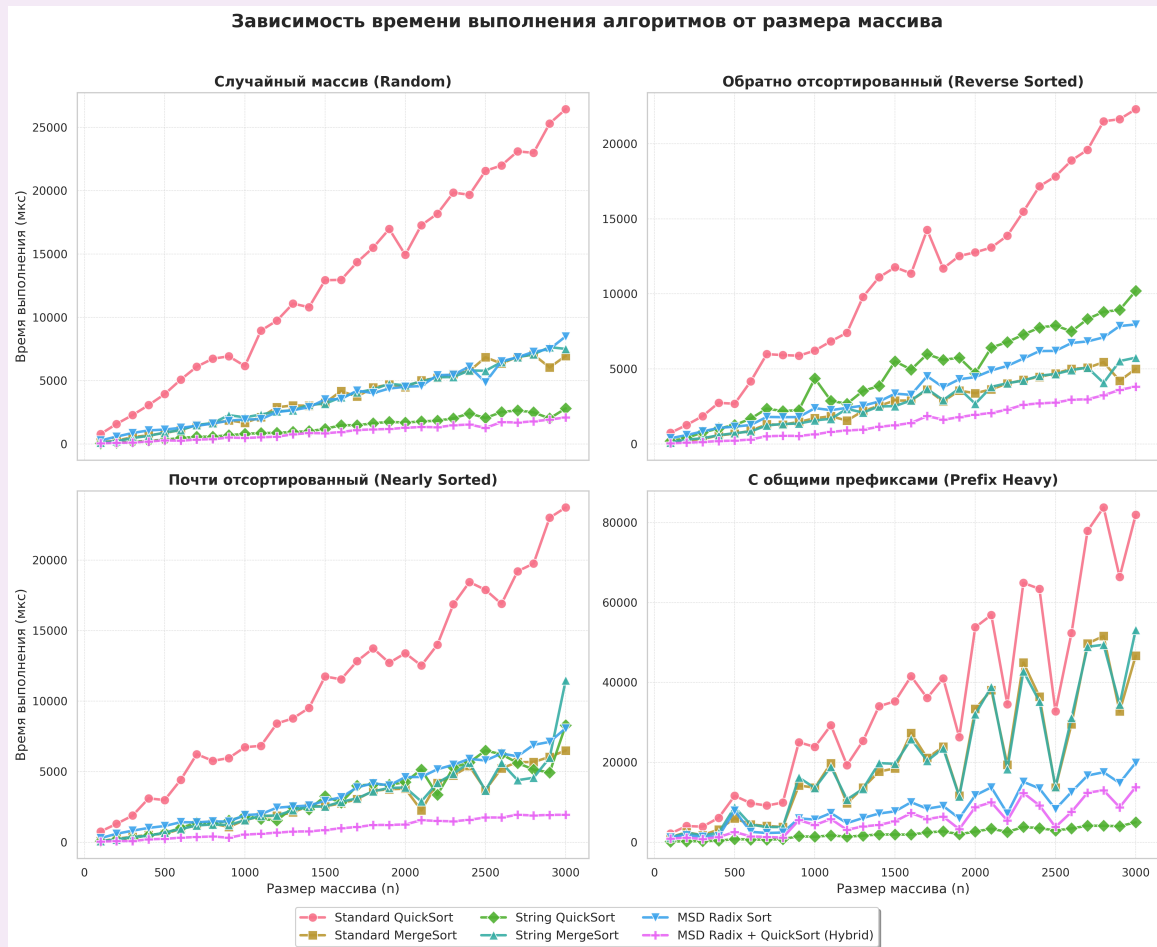
```

1 // -----
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 const int ALPHABET_SIZE = 256;
7 const int THRESHOLD = 74;
8
9 int char_at(const std::string& value, int depth) {
10     if (depth >= static_cast<int>(value.size())) {
11         return -1;
12     }
13     return static_cast<unsigned char>(value[depth]);
14 }
15
16 void string_quick_sort(std::vector<std::string>& values,
17     int left,
18     int right,
19     int depth) {
20     if (left >= right) return;
21
22     int lower = left;
23     int greater = right;
24     int pivot = char_at(values[left], depth);
25     int index = left + 1;
26
27     while (index <= greater) {
28         int current = char_at(values[index], depth);
29
30         if (current < pivot) {
31             std::swap(values[lower++], values[index++]);
32         } else if (current > pivot) {
33             std::swap(values[index], values[greater--]);
34         } else {
35             ++index;
36         }
37     }
38
39     string_quick_sort(values, left, lower - 1, depth);
40     if (pivot >= 0) string_quick_sort(values, lower, greater, depth + 1);
41     string_quick_sort(values, greater + 1, right, depth);
42 }
43
44 void hybrid_sort(std::vector<std::string>& values,
45     std::vector<std::string>& buffer,
46     int left,
47     int right,
48     int depth) {
49     if (right - left <= 1) return;
50
51     if (right - left < THRESHOLD) {
52         string_quick_sort(values, left, right - 1, depth);
53         return;
54     }
55
56     std::vector<int> count(ALPHABET_SIZE + 2, 0);
57
58     for (int index = left; index < right; ++index) {
59         ++count[char_at(values[index], depth) + 2];
60     }
61
62     for (int index = 1; index < static_cast<int>(count.size()); ++index) {
63         count[index] += count[index - 1];
64     }
65
66     std::vector<int> start_positions = count;
67
68     for (int index = left; index < right; ++index) {
69         int bucket = char_at(values[index], depth) + 1;
70         buffer[left + count[bucket]++] = values[index];
71     }
72
73     for (int index = left; index < right; ++index) values[index] = buffer[index];
74
75     for (int bucket = 0; bucket < ALPHABET_SIZE; ++bucket) {
76         int begin = left + start_positions[bucket];
77         int end = left + start_positions[bucket + 1];
78         if (end - begin > 1) hybrid_sort(values, buffer, begin, end, depth + 1);
79     }
80 }

```

6. Результаты экспериментов

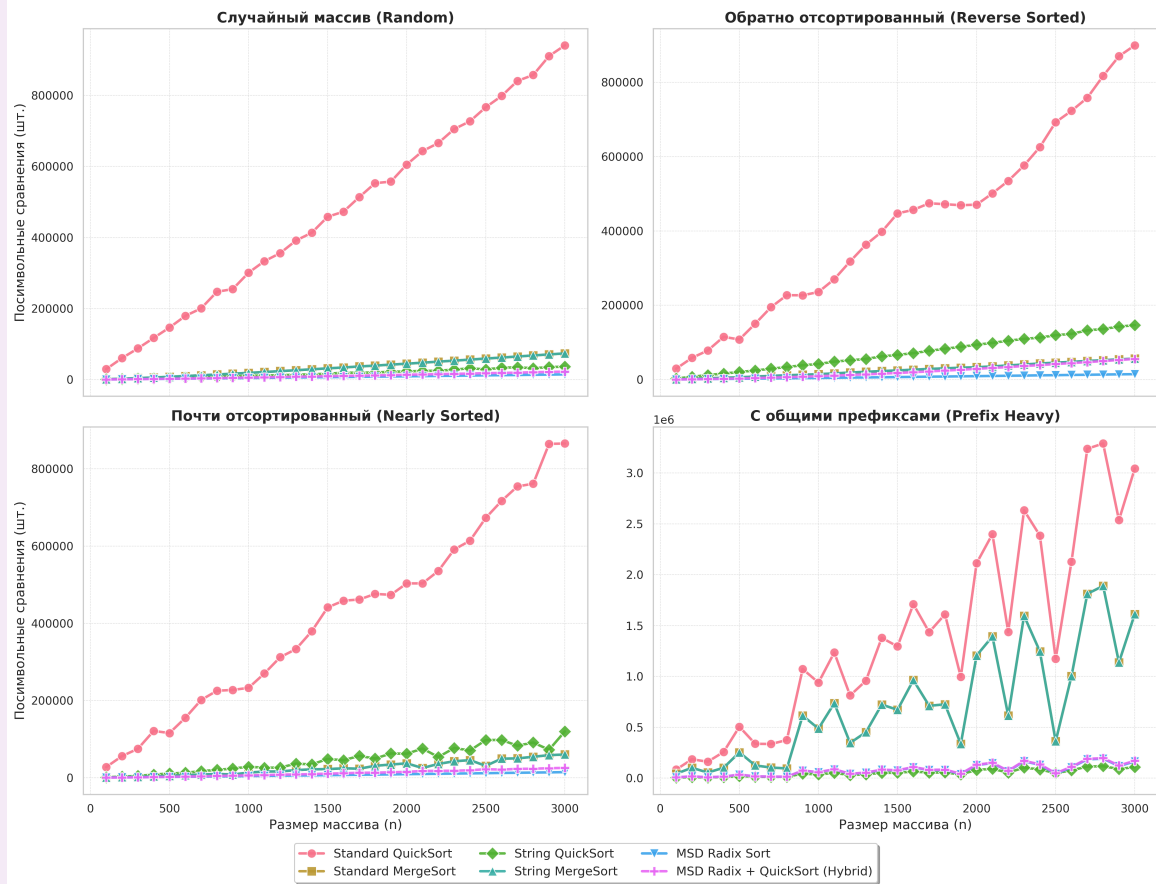
График времени выполнения



Примечание: На графике представлена зависимость среднего времени работы алгоритмов от размера массива для четырех различных типов тестовых данных.

График посимвольных сравнений

Зависимость количества посимвольных сравнений от размера массива



Примечание: График демонстрирует количество посимвольных сравнений. Заметно радикальное сокращение вызовов сравнений у строковых сортировок (особенно на наборе Prefix Heavy).

7. Анализ результатов

Сравнение с теоретическими оценками:

1. **Теория подтверждается:** Стандартные сортировки на строках делают избыточное количество сравнений. В наборах с совпадающими префиксами они заново проверяют каждый префикс.
2. **STRING MERGESORT:** Алгоритм на практике подтвердил сложность $O(D + n \log n)$, где D — просмотренные символы. График количества сравнений явно уходит вниз по сравнению со стандартным MergeSort.
3. **MSD RADIX SORT:** Продемонстрировал почти линейный тренд времени. Отсутствие прямых посимвольных сравнений (использование `char_at` для индексов массивов корзины) делает его недостижимым по метрике сравнений.
4. **Накладные расходы:** Чистый Radix Sort теряет позиции на мелких подмассивах из-за рекурсии. Гибридный подход с переключением на Quick Sort успешно нивелирует этот недостаток.

8. Приложение. Реализация вспомогательных классов

В соответствии с условиями задачи, ниже приведены исходные коды классов `StringGenerator` и `StringSortTester`, разработанных для генерации тестовых наборов и проведения эмпирических замеров производительности.

Листинг кода: Реализация класса `StringGenerator` (и вспомогательного счётчика сравнений `ComparisonCounter`) представлена на следующей странице.

```

1 // -----
2 #include <algorithm>
3 #include <chrono>
4 #include <fstream>
5 #include <iostream>
6 #include <random>
7 #include <string>
8 #include <utility>
9 #include <vector>
10
11 class ComparisonCounter {
12 public:
13     static void Reset() { character_comparisons_ = 0; }
14     static void Add(long long value = 1) { character_comparisons_ += value; }
15     static long long Get() { return character_comparisons_; }
16 private:
17     static long long character_comparisons_;
18 };
19
20 long long ComparisonCounter::character_comparisons_ = 0;
21
22 class StringGenerator {
23 public:
24     explicit StringGenerator(unsigned int seed = 1337)
25         : random_engine(seed),
26           length_distribution(10, 200),
27           character_distribution(0, static_cast<int>(GetAlphabet().size()) - 1) {}
28
29     std::vector<std::string> GenerateRandomDataset(int size, int min_length,
30                                                    int max_length) const {
31         return GenerateBaseDataset(size, min_length, max_length);
32     }
33
34     std::vector<std::string> GenerateReverseSortedDataset(int size,
35                                                           int min_length,
36                                                           int max_length) const {
37         std::vector<std::string> values =
38             GenerateBaseDataset(size, min_length, max_length);
39         std::sort(values.begin(), values.end());
40         std::reverse(values.begin(), values.end());
41         return values;
42     }
43
44     std::vector<std::string> GenerateNearlySortedDataset(int size, int min_length,
45                                                          int max_length) const {
46         std::vector<std::string> values =
47             GenerateBaseDataset(size, min_length, max_length);
48         std::sort(values.begin(), values.end());
49
50         std::uniform_int_distribution<int> swap_count_distribution(
51             1, std::max(1, size / 20));
52         int swap_count = swap_count_distribution(random_engine_);
53
54         std::uniform_int_distribution<int> index_distribution(0, size - 1);
55         for (int index = 0; index < swap_count; ++index) {
56             int first = index_distribution(random_engine_);
57             int second = index_distribution(random_engine_);
58             std::swap(values[first], values[second]);
59         }
60         return values;
61     }
62
63     std::vector<std::string> GeneratePrefixHeavyDataset(int size, int min_length,
64                                                         int max_length) const {
65         std::vector<std::string> values;
66         values.reserve(size);
67         std::uniform_int_distribution<int> prefix_length_distribution(5, 40);
68         int prefix_length = prefix_length_distribution(random_engine_);
69         std::string prefix = RandomString(prefix_length);
70         for (int index = 0; index < size; ++index) {
71             int current_length = RandomLength(min_length, max_length);
72             int suffix_length = std::max(0, current_length - prefix_length);
73             std::string value = prefix;
74             value += RandomString(suffix_length);
75             values.push_back(std::move(value));
76         }
77         return values;
78     }
79
80 private:
81     int RandomLength(int min_length, int max_length) const {
82         std::uniform_int_distribution<int> distribution(min_length, max_length);
83         return distribution(random_engine_);
84     }
85
86     std::string RandomString(int length) const {
87         std::string result;
88         result.reserve(length);
89         const std::string& alphabet = GetAlphabet();
90         for (int index = 0; index < length; ++index) {
91             result.push_back(alphabet[character_distribution_(random_engine_)]);
92         }
93         return result;
94     }
95
96     std::vector<std::string> GenerateBaseDataset(int size, int min_length,
97                                                  int max_length) const {
98         std::vector<std::string> values;
99         values.reserve(size);
100         for (int index = 0; index < size; ++index) {
101             values.push_back(RandomString(RandomLength(min_length, max_length)));

```

```

102     }
103     return values;
104 }
105
106 static const std::string& GetAlphabet() {
107     static const std::string alphabet =
108         "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
109         "abcdefghijklmnopqrstuvwxyz"
110         "0123456789"
111         "!@#%&:~&*()-.,";
112     return alphabet;
113 }
114
115 mutable std::mt19937 random_engine_;
116 mutable std::uniform_int_distribution<int> length_distribution_;
117 mutable std::uniform_int_distribution<int> character_distribution_;
118 };

```

Листинг кода: Реализация класса `StringSortTester` и функции `main` для инициализации замеров представлена на следующей странице.

```

1 // -----
2 class StringSortTester {
3 public:
4     struct Result {
5         std::string algorithm;
6         std::string dataset;
7         int size;
8         double average_time_microseconds;
9         double average_character_comparisons;
10    };
11
12    explicit StringSortTester(int repetitions = 10)
13        : repetitions_(repetitions), generator_(1337) {}
14
15    std::vector<Result> RunAllTests() {
16        std::vector<Result> results;
17        const std::vector<int> sizes = {100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1100, 1200, 1300, 1400, 1500, 1600, 1700, 1800,
18            1900, 2000, 2100, 2200, 2300, 2400, 2500, 2600, 2700, 2800, 2900, 3000};
19
20        int total_tests = static_cast<int>(sizes.size()) * 4 * 6;
21        int current_test = 0;
22
23        for (int size : sizes) {
24            auto random_data = generator_.GenerateRandomDataset(size, 10, 200);
25            auto reverse_data = generator_.GenerateReverseSortedDataset(size, 10, 200);
26            auto nearly_sorted_data = generator_.GenerateNearlySortedDataset(size, 10, 200);
27            auto prefix_heavy_data = generator_.GeneratePrefixHeavyDataset(size, 10, 200);
28
29            const std::vector<std::pair<std::string, std::vector<std::string>>> datasets = {
30                {"random", random_data},
31                {"reverse_sorted", reverse_data},
32                {"nearly_sorted", nearly_sorted_data},
33                {"prefix_heavy", prefix_heavy_data}
34            };
35
36            for (const auto& dataset_entry : datasets) {
37                results.push_back(Benchmark("quick_sort", dataset_entry.first, dataset_entry.second, &RunStandardQuickSort));
38                std::cout << "\rProgress:_" << ++current_test << "/" << total_tests << std::flush;
39
40                results.push_back(Benchmark("merge_sort", dataset_entry.first, dataset_entry.second, &RunStandardMergeSort));
41                std::cout << "\rProgress:_" << ++current_test << "/" << total_tests << std::flush;
42
43                results.push_back(Benchmark("string_quick_sort", dataset_entry.first, dataset_entry.second, &RunStringQuickSort));
44                std::cout << "\rProgress:_" << ++current_test << "/" << total_tests << std::flush;
45
46                results.push_back(Benchmark("string_merge_sort", dataset_entry.first, dataset_entry.second, &RunStringMergeSort));
47                std::cout << "\rProgress:_" << ++current_test << "/" << total_tests << std::flush;
48
49                results.push_back(Benchmark("msd_radix_sort", dataset_entry.first, dataset_entry.second, &RunMsdRadixSort));
50                std::cout << "\rProgress:_" << ++current_test << "/" << total_tests << std::flush;
51
52                results.push_back(Benchmark("msd_radix_quick_sort", dataset_entry.first, dataset_entry.second, &RunHybridSort));
53                std::cout << "\rProgress:_" << ++current_test << "/" << total_tests << std::flush;
54            }
55            std::cout << std::endl;
56            return results;
57        }
58
59        void SaveCsv(const std::string& path, const std::vector<Result>& results) const {
60            std::ofstream output(path);
61            output << "algorithm,dataset,size,average_time_microseconds,average_character_comparisons\n";
62            for (const Result& result : results) {
63                output << result.algorithm << ',' << result.dataset << ',' << result.size << ',' << result.average_time_microseconds << ',' <<
64                    result.average_character_comparisons << '\n';
65            }
66
67        private:
68            using SortFunction = void (*)(std::vector<std::string>&);
69            static void RunStandardQuickSort(std::vector<std::string>& values) { if (!values.empty()) StandardQuickSort(values, 0, static_cast<int>
70                >(values.size() - 1)); }
71            static void RunStandardMergeSort(std::vector<std::string>& values) { std::vector<std::string> buffer(values.size()); StandardMergeSort(
72                values, buffer, 0, static_cast<int>(values.size())); }
73            static void RunStringQuickSort(std::vector<std::string>& values) { if (!values.empty()) StringQuickSort(values, 0, static_cast<int>(
74                values.size() - 1, 0)); }
75            static void RunStringMergeSort(std::vector<std::string>& values) { std::vector<std::string> buffer(values.size()); StringMergeSort(
76                values, buffer, 0, static_cast<int>(values.size())); }
77            static void RunMsdRadixSort(std::vector<std::string>& values) { std::vector<std::string> buffer(values.size()); MsdRadixSort(values,
78                buffer, 0, static_cast<int>(values.size()), 0); }
79            static void RunHybridSort(std::vector<std::string>& values) { std::vector<std::string> buffer(values.size()); HybridSort(values, buffer
80                , 0, static_cast<int>(values.size()), 0); }
81
82            Result Benchmark(const std::string& algorithm, const std::string& dataset, const std::vector<std::string>& values, SortFunction
83                sort_function) const {
84                double total_time = 0.0;
85                double total_comparisons = 0.0;
86                for (int repetition = 0; repetition < repetitions_; ++repetition) {
87                    std::vector<std::string> copy = values;
88                    ComparisonCounter::Reset();
89                    const auto start = std::chrono::steady_clock::now();
90                    sort_function(copy);
91                    const auto finish = std::chrono::steady_clock::now();
92                    total_time += std::chrono::duration_cast<std::chrono::duration<double, std::micro>>(finish - start).count();
93                    total_comparisons += static_cast<double>(ComparisonCounter::Get());
94                }
95                return Result{algorithm, dataset, static_cast<int>(values.size()), total_time / repetitions_, total_comparisons / repetitions_};
96            }
97
98            int repetitions_;
99            StringGenerator generator_;
100    };

```

```
93 };
94
95 int main() {
96     StringSortTester tester(10);
97     auto results = tester.RunAllTests();
98     tester.SaveCsv("results.csv", results);
99     std::cout << "Saved_results.csv" << std::endl;
100     return 0;
101 }
```

9. Общие итоги

Общие итоги исследования:

- STRING QUICK SORT эффективно работает на строках с длинными префиксами.
- STRING MERGESORT значительно уменьшает количество повторных сравнений.
- MSD RADIX SORT показывает отличную, близкую к линейной сложность.
- Гибридный алгоритм MSD RADIX + QUICK SORT демонстрирует лучшие результаты.
- Специализированные алгоритмы на практике превосходят стандартные сортировки строк.

Дополнительная информация

ID посылок Codeforces:

A1m: 375977803

A1q: 375978016

A1r: 375978260

A1rq: 375978506

Ссылка на репозиторий: <https://github.com/danilprakh0v/a1-analysis-string-sorts>